[2]

Local Repositories

In the last chapter, you learned some command line basics and you prepared to work with Git by downloading it and configuring some settings.
In this chapter, you will turn the rainbow project directory you created in Chapter 1 into a Git repository. You will also learn about four important areas when working with Git: the working directory, the staging area, the commit history, and the local repository. To help you visualize how each of these areas works together, we will build a Git Diagram that will include a representation of each area.
Finally, at the end of this chapter you will add the first file to the rainbow project directory, and in the process learn about untracked and tracked files. Let's get started!

Current Setup

At the start of this chapter you should have:

- Downloaded or updated Git on your computer. (You will need a version equal to or greater than 2.28.)
- Created an empty project directory called rainbow on your desktop.
- Opened a command line window and navigated into the rainbow directory.
- Decided on a text editor to use and opened the rainbow project directory in a text editor window.
- Set the user.name and user.email global Git configuration variables to your name and email address.

Introducing Repositories

A *repository* (also known as a *repo*) is how we refer to a project version controlled by Git. In reality, there are two types of repositories:

- A *local repository* is a repository that is stored on a computer.
- A *remote repository* is a repository that is hosted on a hosting service.

A *hosting service* is a company that provides hosting for projects using Git. As of the time of writing, the main hosting services are GitHub (*https://github.com*), GitLab (*https://about.gitlab.com*), and Bitbucket (*https://bitbucket.org/product*).

In the first part of this book, up through Chapter 5, you will learn about and work only with local repositories. In the second part of the book, from Chapter 6 onward, you will learn how to work with remote repositories as well.

Now that we've distinguished between local and remote repositories, you'll learn about initializing a local repository.

Initializing a Local Repository

A local repository is represented by a hidden directory called .git that exists within a project directory. It contains all the data on the changes that have been made to the files in a project.

To turn a project directory into a local repository you have to *initialize*, or create, the repository. When you initialize a repository, the .git directory is automatically created inside the project directory. Because the .git directory is a hidden directory, you won't be able to see it unless you explicitly make hidden files and directories visible.

[NOTE]

You should never touch any files or directories inside your .git directory. Doing this could have undesirable consequences for your repository. You should never delete this directory unless you want to delete your repository.

In Follow Along 2-1, you will make sure you have your settings configured to be able to view hidden files and directories. (If you need a reminder of how to do this, refer to "Viewing the Contents of Directories" on page 10.) You will then check both in your filesystem window and your command line window to see if there are any visible or hidden files or directories in the rainbow project directory.

[FOLLOW ALONG 2-1]

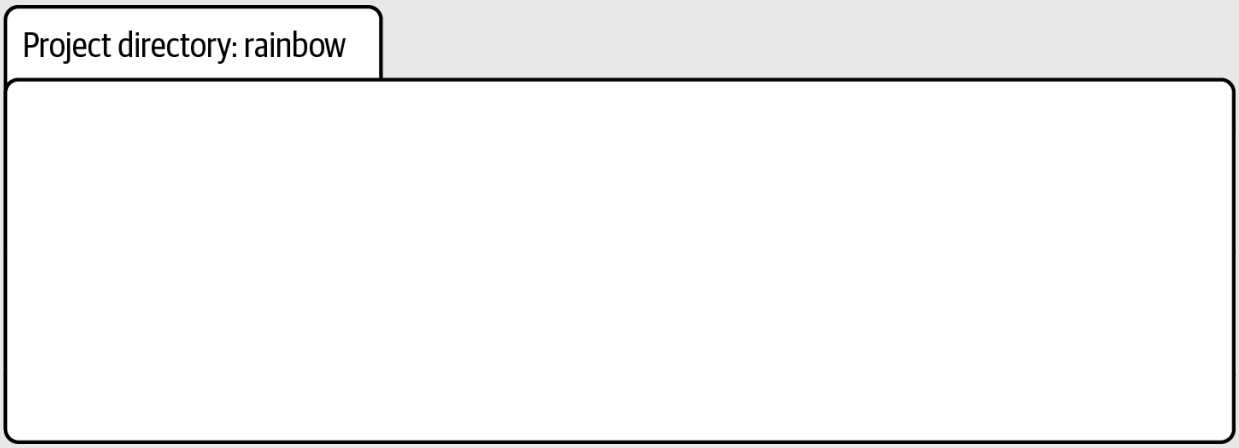
1. Make hidden files and directories visible in your filesystem.
2. Open the rainbow project directory in a filesystem window and look at its contents. There should be no visible or hidden files or directories.
3. rainbow $ **ls -a**
   . ..

What to notice:

- In step 2, in the filesystem window, you can see that the rainbow project directory is empty.
- In step 3, in the command line window, you see that the rainbow project directory is empty.

To prepare to build the Git Diagram, we will create a representation of the empty rainbow project directory as seen in Visualize it 2-1.

[VISUALIZE IT 2-1]



A representation of the empty rainbow project directory that will contain the Git Diagram

To initialize a Git repository, you use the git init command. Your current directory must be the project directory you want to turn into a repository when you execute this command.

Normally Git users initialize a Git repository by using just the git init command with no additional options; however, in the Rainbow project, you will initialize the repository by using the git init command with the -b option (which is short for --initial-branch) and pass in the name main. We will cover what branches are in more depth in Chapter 4. For now, all you need to know is that by default Git will create a branch called master when you initialize a new local repository. From Git version 2.28 onwards, the name of the initial branch is configurable. I have chosen to use the name main instead of master in this book, because "master" is not considered inclusive terminology. We will discuss this topic more in "A Bit of Git History: master and main" on page 48.

[NOTE]

If you decide that you want the initial branch in all of your repositories to have a name other than “master,” then you may set a variable called init.defaultBranch in your global configuration file. The process is similar to how you set the user.name and user.email variables in “Setting Git Configurations” on page 17. If the init.defaultBranch variable is defined, you can initialize Git repositories with the git init command, and the initial branch will have the name that is defined in the configuration.

[SAVE THE COMMAND]

git init

Initialize a Git repository

git init -b <branch_name>

Initialize a Git repository and set the name for the initial branch to be <branch_name>

Go to Follow Along 2-2 to turn your rainbow project directory into a Git repository.

[FOLLOW ALONG 2-2]

1. To see the .git directory being created, make sure you have a view of the contents of your rainbow project directory in a filesystem window with hidden files and directories enabled.

2. rainbow $ **git init -b main**
   Initialized empty Git repository in
   /Users/annaskoulikari/desktop/rainbow/.git/

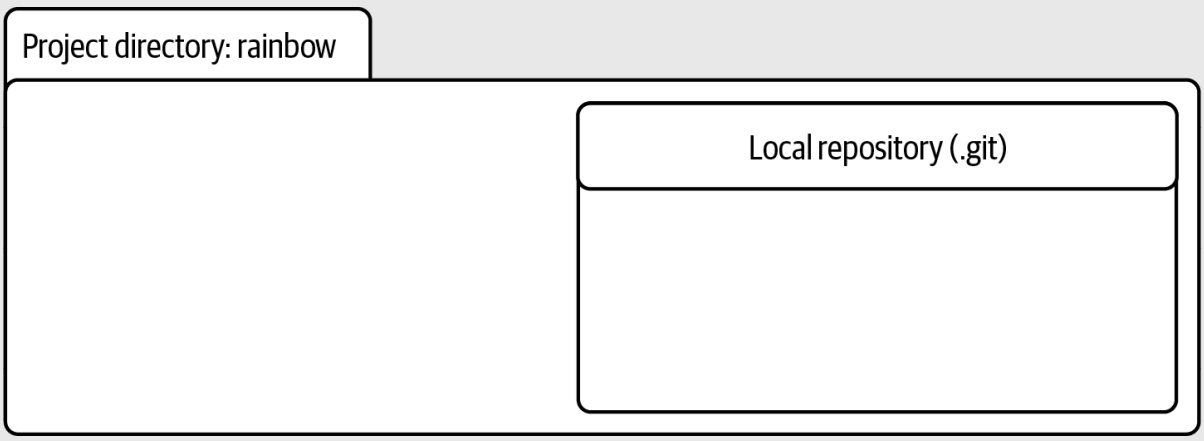
3. Go to the rainbow project directory in the filesystem window and look at the .git directory that was just created. Open the .git directory to view the contents inside.

What to notice:

- Git created the .git directory inside the rainbow project directory.

The first area for which we will add a representation in the Git Diagram is the local repository, represented by the .git directory itself. This can be seen in Visualize it 2-2.

[VISUALIZE IT 2-2]



The Git Diagram with a representation of the local repository

Inside the .git directory are various files and directories. Some of them represent the areas of Git that you are going to learn about next. In the following section, we will add representations of these to the Git Diagram as well. As you continue working in the rainbow repository, more files and directories will be created in the .git directory; you'll learn about some of these as we go along.

Figure 2-1 shows an example of the contents of a .git directory in a newly initialized repository.

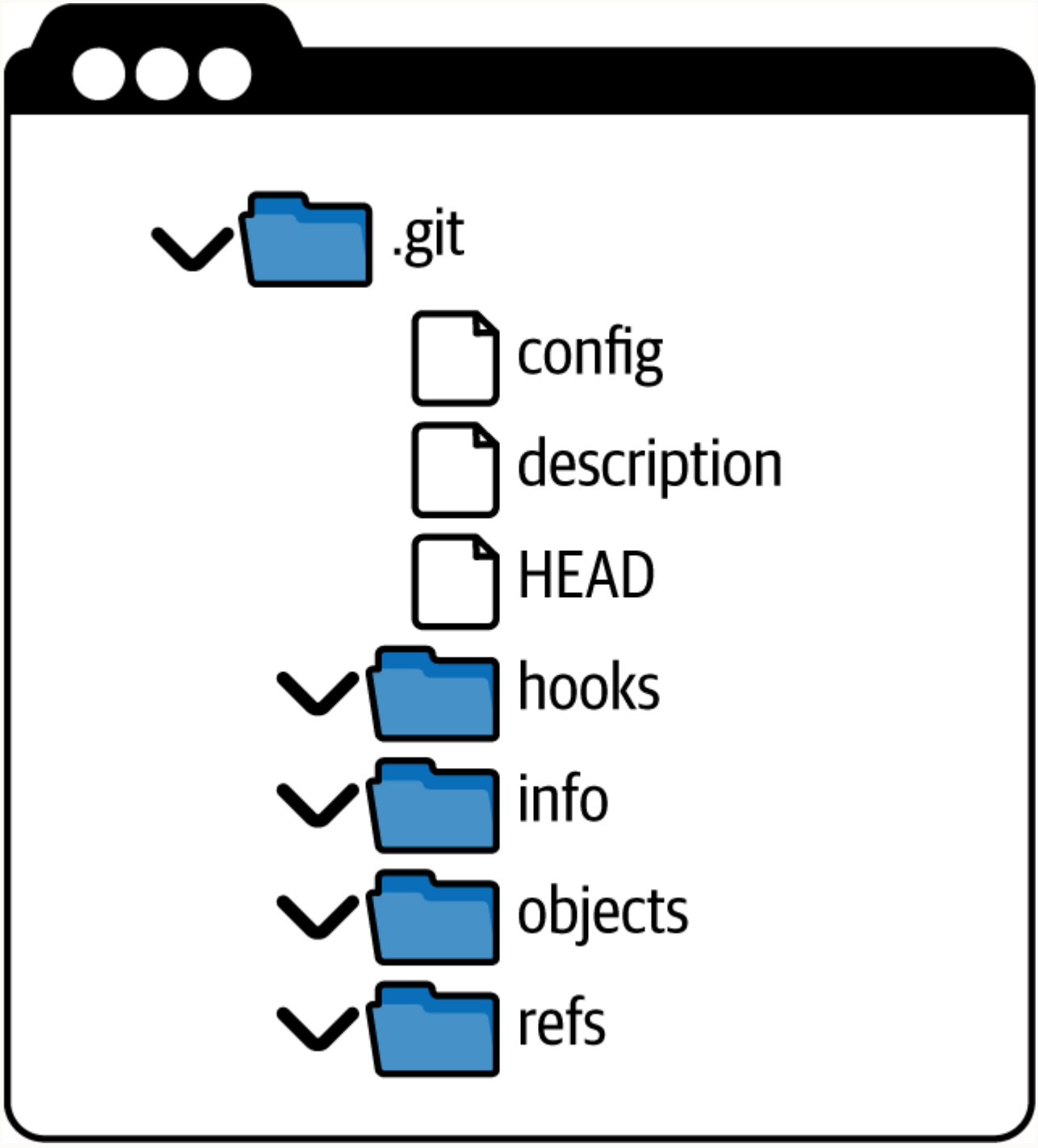


FIGURE 2-1

The contents of a .git directory right after executing the git init command

Next, let's explore the different areas you'll interact with when working with Git.

The Areas of Git

There are four important areas to be aware of when you are working with Git:

- Working directory
- Staging area
- Commit history
- Local repository

You learned about the local repository in the previous section. In this section we will cover the rest of the areas and how they all relate to one another, and we will continue building the Git Diagram. Let's start with the working directory.

INTRODUCING THE WORKING DIRECTORY

The *working directory* contains the files and directories in the project directory that represent one version of a project. It is sort of like a workbench. It is where you add, edit, and delete files and directories.

Let's take a look at Example Book Project 2-1 to explore how files are managed in the working directory.

EXAMPLE BOOK PROJECT 2-1

Suppose the Book project that I am working on has 10 chapters, and I have 10 text files, one for each chapter: chapter_one.txt, chapter_two.txt, and so on.

To add each of these chapter files to my project, I would create these files in the working directory.

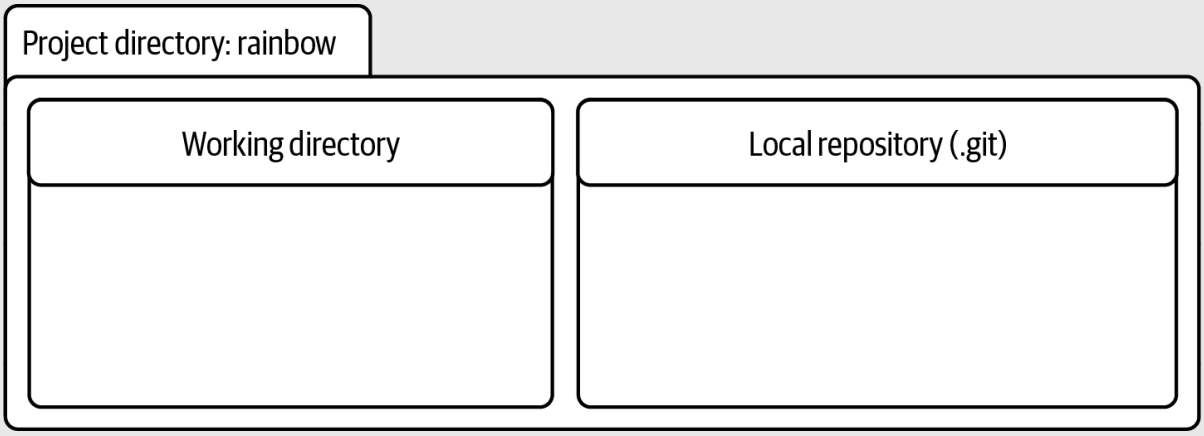
If I wanted to make any changes to the content of those chapters, I would start by editing the files in the working directory.

And finally, if I decided I wanted to remove an entire chapter of my book, the first step would be to delete the corresponding file in the working directory.

From Example Book Project 2-1, you can conclude that the working directory is where you make all the modifications to the content of a project.

In the case of the rainbow repository, the working directory is currently empty. To continue building the Git Diagram, we will add a representation of the working directory to it. This is illustrated in Visualize it 2-3.

[VISUALIZE IT 2-3]



The Git Diagram with a representation of the working directory and local repository

In Visualize It 2-3, the rainbow project directory contains the local repository within it. However, it is important to know that when people refer to a project version controlled with Git, they will typically refer to the project directory as the repository. For example, in our case, we will make references to "the rainbow repository."

Within the local repository, there are two important areas we want to explore further: the staging area and the commit history. We'll take a look at those next and also discuss the concept of a commit in a little more detail.

INTRODUCING THE STAGING AREA

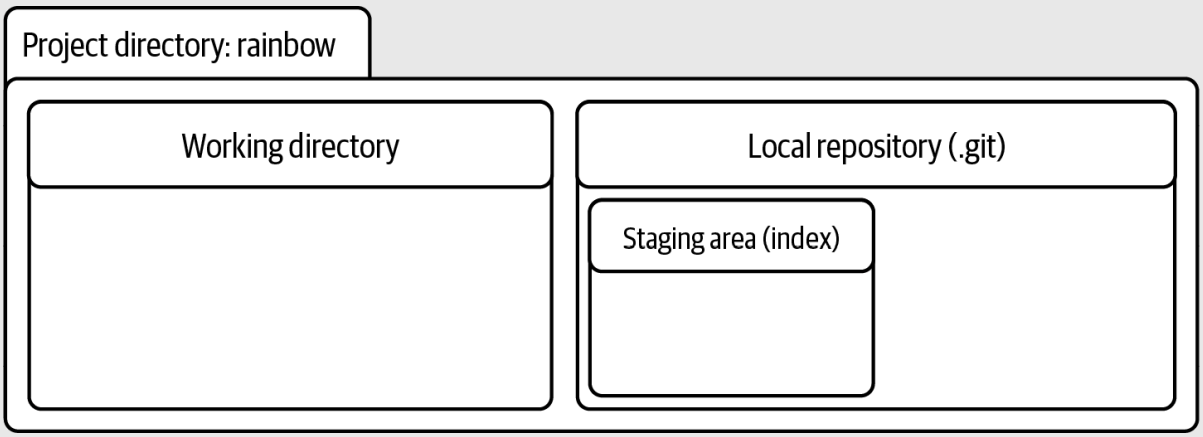
The *staging area* is similar to a rough draft space. It is where you can add and remove files, when you are preparing what you want to include in the next saved version of your project (your next commit). The staging area is represented by a file in the .git directory called index.

[NOTE]

The index file is created only if you have added at least one file to the staging area in your project. In the rainbow project directory, you have not yet added any files to the staging area; therefore, the index file is not yet visible in the .git directory. In Chapter 3, you will add a file to the staging area and you will see the index file being created.

You will learn more about why the staging area is so helpful when you practice adding a file to it in Chapter 3. For now, we will add a representation of it inside the local repository in the Git Diagram, as shown in Visualize it 2-4.

[VISUALIZE IT 2-4]



The Git Diagram with a representation of the working directory, staging area, and local repository

The Git Diagram now includes representations of the working directory, the staging area, and the local repository. The final area I want to introduce is the commit history, but before I do that, let's properly explore the concept of a commit.

WHAT IS A COMMIT?

A *commit* in Git is basically one version of a project. You can think of it as a snapshot of a project, or a standalone version of a project that contains references to all the files that

are part of that commit.

Every commit has a *commit hash* (sometimes called a *commit ID*). This is a unique 40-character hash composed of letters and numbers that acts like a name for the commit, providing a way to refer to it.

An example of a commit hash is 51dc6ecb327578cca503abba4a56e8c18f3835e1.

In reality, you only need the first seven characters of a commit hash to refer to a commit. So, for the example hash just given, you can just use 51dc6ec to refer to the commit.

[NOTE]

Since commit hashes are unique, your commit hashes in the Rainbow project will be different than the ones shown in this book. Keep this in mind while going through the Follow Along exercises in the book.

Now that you have an idea of what commits are, let's introduce the final area that we will add to our Git Diagram, the commit history.

INTRODUCING THE COMMIT HISTORY

The *commit history* is where you can think of your commits existing. It is represented by the objects directory inside the .git directory. To understand the commit history in depth we would have to dive into the internals of Git, but that is a complex topic that you don't need to know about when learning the basics of how to use Git. For our purposes, all you need to know is that every time you make a commit, it is saved in the commit history.

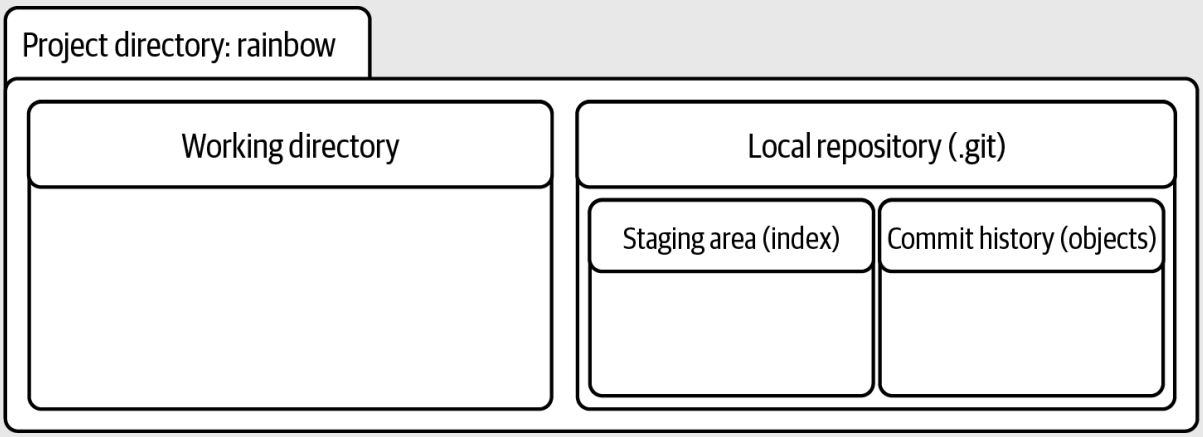
Go to Follow Along 2-3 to identify the commit history in the rainbow repository.

[FOLLOW ALONG 2-3]

1. In your filesystem window, inside your rainbow project directory, look inside the .git directory and find the objects directory.

In Visualize it 2-5, we add a representation of the commit history to the Git Diagram inside the local repository. This completes the Git Diagram.

[VISUALIZE IT 2-5]



The complete Git Diagram with representations of the working directory, staging area, commit history, and local repository

Now that we have a complete Git Diagram showing the most important areas when working with Git, let's add the first file to the Rainbow project and use a text editor to edit it.

Adding a File to a Git Project

As mentioned earlier in the book, throughout this learning journey you will work on a project in which you list the colors of the rainbow and some colors that are not part of the rainbow. Every time you add a color, you will make a commit to keep track of how the project progresses.

Your first step will be to create a file called rainbowcolors.txt, in which you will list the colors. As mentioned in "Creating a Directory" on page 14, it is important that the filename does not include spaces. Then, using the text editor that you prepared in Chapter 1, you will add "Red is the first color of the rainbow." on line 1 in that file. Go to Follow Along 2-4 to create and edit your file.

[FOLLOW ALONG 2-4]

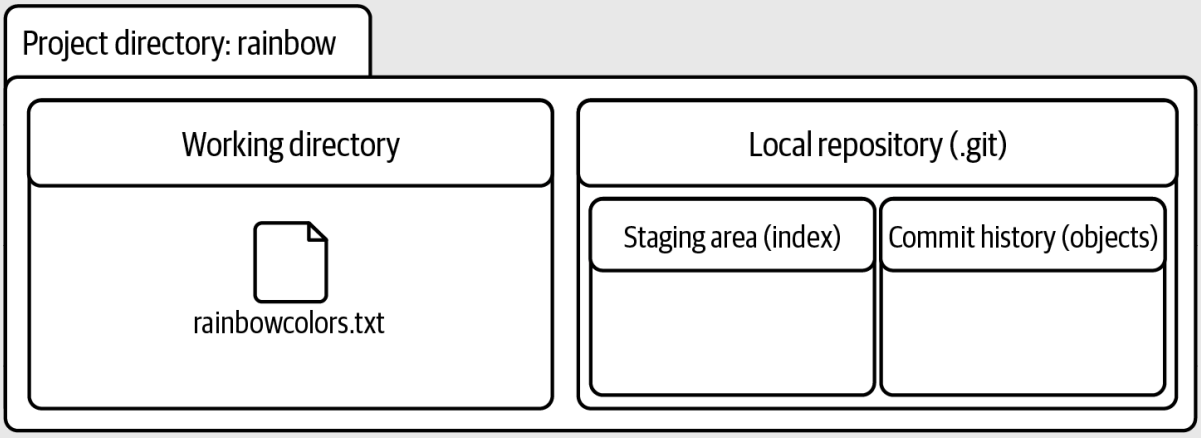
1. Use your text editor to create a file called rainbowcolors.txt inside your rainbow project directory.
2. On line 1, add "Red is the first color of the rainbow." and save the file.

What to notice:

- The rainbowcolors.txt file is inside the rainbow project directory; therefore, it is in the working directory.

Even though the rainbowcolors.txt file is in the working directory, it is not part of your repository. It has not been added to the staging area and it has not yet been included in a commit in the commit history. We illustrate this in Visualize It 2-6.

[VISUALIZE IT 2-6]



The rainbow project directory after you add the rainbowcolors.txt file to the working directory

Since the rainbowcolors.txt file is not yet in your repository, it is an untracked file. An *untracked file* is a file in the working directory that Git is not version controlling. It has never been added to the staging area and it has never been included in a commit; therefore, it is not part of the repository.

Once you add a file to the staging area and include it in a commit, the file becomes a *tracked file*. This is a file that is version controlled (in other words, a file that Git tracks).

Every new file in a project version controlled by Git needs to be explicitly added to the staging area and then included in a commit in order to become a tracked file. You will carry out these steps in Chapter 3.

[NOTE]

In the Preface, I mentioned that creating and editing files listing the colors of the rainbow and some colors that are not part of the rainbow is not a very realistic example of a project version controlled with Git. Recall that the aim of this project is to keep the learning journey simple, so that you can focus on how Git works instead of the contents of the files you are editing. The changes you will be making to files in real projects will look very different.

Summary

In this chapter, you turned your rainbow project directory into a repository by initializing the repository. We also built a Git Diagram visualizing the four important areas of Git: the working directory, the staging area, the commit history, and the local repository.

You added your first file to your rainbow project directory, and you learned about the important distinction between untracked files and tracked files.

In the next chapter, we will walk through the steps involved in making a commit. Along the way, we will touch upon how each of the areas in the Git Diagram is involved in the commiting process.

Previous chapter

‹ Chapter 1. Git and the Command Line

Next chapter

Chapter 3. Making a Commit ›